

# TW07: Interacting with Databases

Five challenges | 20 marks each | 100 marks total

|                 |  |
|-----------------|--|
| Student name    |  |
| Student ID      |  |
| Class / group   |  |
| Submission date |  |

**Weekly purpose**

Use SQLAlchemy with SQLite to create schemas, perform CRUD operations, run parameterised queries, understand joins, and complete a small database analysis.

**SUBMISSION RULES**

- Complete work in the corresponding Jupyter notebook.
- Write your own code and explanations; no solutions are supplied.
- Keep supplied data unchanged and work on copies.
- Restart the kernel and run all cells before submission.
- Submit the notebook with outputs visible and assistance acknowledged.

**MARKING RUBRIC FOR EACH CHALLENGE**

| Assessment area                  | Marks |
|----------------------------------|-------|
| Correct method and working code  | 8     |
| Validation and quality checks    | 4     |
| Explanation and interpretation   | 4     |
| Reproducibility and presentation | 4     |
| Total per challenge              | 20    |

## Challenge 1: Create a safe practice database

### Scenario

A training project needs an isolated SQLite database and a clearly defined students table.

### INSTRUCTIONS

1. Confirm that SQLAlchemy is installed and report its version.
2. Create an in-memory SQLite engine.
3. Define a students table with student\_id, name, course, and score.
4. Choose appropriate primary key, text, and numeric types.
5. Create the table and inspect its column names.

### REQUIRED SUBMISSION

- A package/version check and working engine.
- A valid SQLAlchemy table schema.
- A displayed column inventory and short schema explanation.

**20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4**

## Challenge 2: Perform create, read, update, and delete operations

### Scenario

A course administrator must maintain student records accurately.

### INSTRUCTIONS

1. Create a fresh in-memory engine and students table.
2. Insert at least five rows in one transaction.
3. Select all rows and then filter students with scores of at least 70.
4. Update exactly one student's score and verify the change.
5. Delete exactly one identified row and verify the final row count.

### REQUIRED SUBMISSION

- Working CRUD statements with transaction handling.
- Before-and-after query results.
- Assertions for updated value and final row count.

**20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4**

### Challenge 3: Run a parameterised SQL query

#### Scenario

A report must filter by a user-supplied minimum score without building unsafe SQL strings.

#### INSTRUCTIONS

1. Create and populate a small students table.
2. Write a textual SELECT statement using a named parameter such as :minimum\_score.
3. Execute it with minimum scores of 60 and 80.
4. Return column names and rows in a readable form.
5. Explain why parameters are safer than inserting user text into an SQL string.

#### REQUIRED SUBMISSION

- One parameterised statement and two executions.
- Readable results with correct filtering.
- A concise SQL-injection safety explanation.

**20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4**

### Challenge 4: Compare inner, left, and right-style joins

#### Scenario

Customer and order tables contain matched and unmatched keys that must not be hidden.

#### INSTRUCTIONS

1. Create customers and orders tables from the supplied CSV files.
2. Run an inner join and explain which rows remain.
3. Run a left join that preserves every customer.
4. Produce a right-style result by starting from orders and left-joining customers.
5. List customers without orders and orders without a valid customer.

#### REQUIRED SUBMISSION

- Three labelled join results.
- Two unmatched-key audit tables.
- Row-count checks and a join-behaviour explanation.

**20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4**

## Challenge 5: Complete a customer-order database report

### Scenario

A manager wants customer activity and revenue summarised from related tables.

### INSTRUCTIONS

1. Load customers.csv and orders.csv into an in-memory SQLite database through SQLAlchemy.
2. Join the tables and calculate order count, total revenue, and average order value by customer.
3. Preserve customers with no orders and replace only their aggregate values appropriately.
4. Sort the report by total revenue and identify the leading customer.
5. Validate source row counts, joined detail rows, unmatched keys, and aggregate totals.

### REQUIRED SUBMISSION

- A complete one-row-per-customer report.
- At least five validation checks.
- A finding, limitation, and reproducibility note.

**20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4**

## Final submission checklist

- ☐ All five challenges are complete.
- ☐ The notebook runs from top to bottom without errors.
- ☐ Every output is clearly labelled.
- ☐ Important results include written explanations.
- ☐ Validation checks are included.
- ☐ Supplied raw data has not been overwritten.
- ☐ Sources and assistance are acknowledged.

### Student declaration

I confirm that this submission is my work, that I have acknowledged sources and assistance, and that the displayed outputs were produced by the submitted code.

Signature: \_\_\_\_\_ Date: \_\_\_\_\_